Clustering Lab: K-Means + PCA on Real Data (Wine Chemistry)

Goal: Use **unsupervised learning** (K-Means) to discover meaningful groups in real chemical measurements of wines.
You'll also use **PCA** to (i) visualize high-dimensional data in 2D and (ii) check if dimensionality reduction helps clustering.

Real-world framing: A quality-control team wants to **segment wines by their chemical profile** to tailor production and tasting protocols — without relying on labels.

What you'll do

- Load the **Wine** dataset (13 chemical features).
- Scale features and explore data quickly.
- Use **PCA** for 2D visualization and to understand variance explained.
- Run **K-Means**; select **k** via elbow & silhouette.
- Evaluate cluster quality with internal metrics and (optionally) compare to known cultivars (external metric, for teaching only).
- Interpret clusters by inspecting centroids in the original units.

Author: Matteo Cercola

In [1]:
```
%pip install numpy pandas matplotlib seaborn openpyxl
%pip install ipywidgets

# === Setup & Imports ===
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davie
from sklearn.metrics.cluster import contingency_matrix

# For optional external validation mapping (Hungarian algorithm)
from scipy.optimize import linear_sum_assignment
```

```
Requirement already satisfied: numpy in /usr/local/python/3.14.2/lib/python
3.14/site-packages (2.5.4)
Requirement already satisfied: pandas in /usr/local/python/3.14.2/lib/python
3.14/site-packages (3.1.0)
Requirement already satisfied: matplotlib in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (3.11.2)
Requirement already satisfied: seaborn in /usr/local/python/3.14.2/lib/pytho
n3.14/site-packages (0.13.2)
Requirement already satisfied: openpyxl in /usr/local/python/3.14.2/lib/pyth
on3.14/site-packages (3.1.5)
Requirement already satisfied: python-dateutil>=2.9.0 in /home/codespace/.lo
cal/lib/python3.14/site-packages (from pandas) (3.0.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/python/3.14.2/
lib/python3.14/site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/python/3.14.
2/lib/python3.14/site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/li
b/python3.14/site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in /usr/local/python/3.14.2/lib/pyt
hon3.14/site-packages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in /usr/local/python/3.14.2/lib/
python3.14/site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: et-xmlfile in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.6 in /home/codespace/.local/lib/python
3.14/site-packages (from python-dateutil>=2.9.0->pandas) (1.17.0)
Requirement already satisfied: tzdata in /home/codespace/.local/lib/python3.
14/site-packages (from python-dateutil>=2.9.0->pandas) (2026.6)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: ipywidgets in /usr/local/python/3.14.2/lib/py
thon3.14/site-packages (8.1.9)
Requirement already satisfied: comm>=0.1.3 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in /home/codespace/.local/lib/
python3.14/site-packages (from ipywidgets) (9.17.1)
Requirement already satisfied: traitlets>=4.3.1 in /home/codespace/.local/li
b/python3.14/site-packages (from ipywidgets) (5.16.1)
Requirement already satisfied: widgetsnbextension~=4.0.16 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~=3.0.17 in /usr/local/pyth
on/3.14.2/lib/python3.14/site-packages (from ipywidgets) (3.0.17)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in /home/codes
pace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
(1.1.1)
Requirement already satisfied: jedi>=0.18.2 in /home/codespace/.local/lib/py
thon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.20.1)
Requirement already satisfied: matplotlib-inline>=0.1.6 in /home/codespace/.
local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.2.2)
Requirement already satisfied: pexpect>4.6 in /home/codespace/.local/lib/pyt
hon3.14/site-packages (from ipython>=6.1.0->ipywidgets) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /home/codesp
ace/.local/lib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets)
```

```
(3.0.53)
Requirement already satisfied: psutil>=7 in /home/codespace/.local/lib/pytho
n3.14/site-packages (from ipython>=6.1.0->ipywidgets) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in /home/codespace/.local/li
b/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (2.21.0)
Requirement already satisfied: stack_data>=0.6.0 in /home/codespace/.local/l
ib/python3.14/site-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth>=0.1.4 in /home/codespace/.local/lib/
python3.14/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0
->ipywidgets) (0.9.4)
Requirement already satisfied: parso<0.9.0,>=0.8.7 in /home/codespace/.loca
l/lib/python3.14/site-packages (from jedi>=0.18.2->ipython>=6.1.0->ipywidget
s) (0.8.7)
Requirement already satisfied: ptyprocess>=0.5 in /home/codespace/.local/li
b/python3.14/site-packages (from pexpect>4.6->ipython>=6.1.0->ipywidgets)
(0.7.0)
Requirement already satisfied: executing>=1.2.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (2.3.0)
Requirement already satisfied: asttokens>=2.1.0 in /home/codespace/.local/li
b/python3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidge
ts) (3.0.2)
Requirement already satisfied: pure-eval in /home/codespace/.local/lib/pytho
n3.14/site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.
2.4)
Note: you may need to restart the kernel to use updated packages.
```

1. Load Real Data & Define the Problem

We use the classic **Wine** dataset (chemical analysis of wines from 3 cultivars). Although we *have* labels (cultivar IDs), we will **not** use them to train K-Means — they are only for **external validation** at the end.

Business-style problem: *Segment wines into groups based on their chemical profiles so that the quality-control team can design differentiated protocols.*

In [2]:
```
# Load the dataset
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y_true = pd.Series(wine.target, name="cultivar")  # kept aside for optional

print("Shape:", X.shape)
X.head()
```

```
Shape: (178, 13)
```

Out[2]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flava
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

In [3]:
```
X.columns
```

Out[3]:
```
Index(['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium',
       'total_phenols', 'flavanoids', 'nonflavanoid_phenols',
       'proanthocyanins', 'color_intensity', 'hue',
       'od280/od315_of_diluted_wines', 'proline'],
      dtype='str')
```

2. Feature Scales & Summary

Clustering is distance-based. **Always check scales** and **standardize** features when units differ.

In [4]:
```
display(X.describe().T)
```

	count	mean	std	min	25%
alcohol	178.0	13.000618	0.811827	11.03	12.3625
malic_acid	178.0	2.336348	1.117146	0.74	1.6025
ash	178.0	2.366517	0.274344	1.36	2.2100
alcalinity_of_ash	178.0	19.494944	3.339564	10.60	17.2000
magnesium	178.0	99.741573	14.282484	70.00	88.0000
total_phenols	178.0	2.295112	0.625851	0.98	1.7425
flavanoids	178.0	2.029270	0.998859	0.34	1.2050
nonflavanoid_phenols	178.0	0.361854	0.124453	0.13	0.2700
proanthocyanins	178.0	1.590899	0.572359	0.41	1.2500
color_intensity	178.0	5.058090	2.318286	1.28	3.2200
hue	178.0	0.957449	0.228572	0.48	0.7825
od280/od315_of_diluted_wines	178.0	2.611685	0.709990	1.27	1.9375
proline	178.0	746.893258	314.907474	278.00	500.5000

3. Preprocessing: Standardize Features

We use `StandardScaler` so that each feature has zero mean and unit variance, preventing larger-scale features from dominating distances.

In [5]:
```
scaler = StandardScaler()
X_std = scaler.fit_transform(X)
X_std[:3]
```

Out[5]:
```
array([[ 1.51861254, -0.5622498 ,  0.23205254, -1.16959318,  1.91390522,
         0.80899739,  1.03481896, -0.65956311,  1.22488398,  0.25171685,
         0.36217728,  1.84791957,  1.01300893],
       [ 0.24628963, -0.49941338, -0.82799632, -2.49084714,  0.01814502,
         0.56864766,  0.73362894, -0.82071924, -0.54472099, -0.29332133,
         0.40605066,  1.1134493 ,  0.96524152],
       [ 0.19687903,  0.02123125,  1.10933436, -0.2687382 ,  0.08835836,
         0.80899739,  1.21553297, -0.49840699,  2.13596773,  0.26901965,
         0.31830389,  0.78858745,  1.39514818]])
```

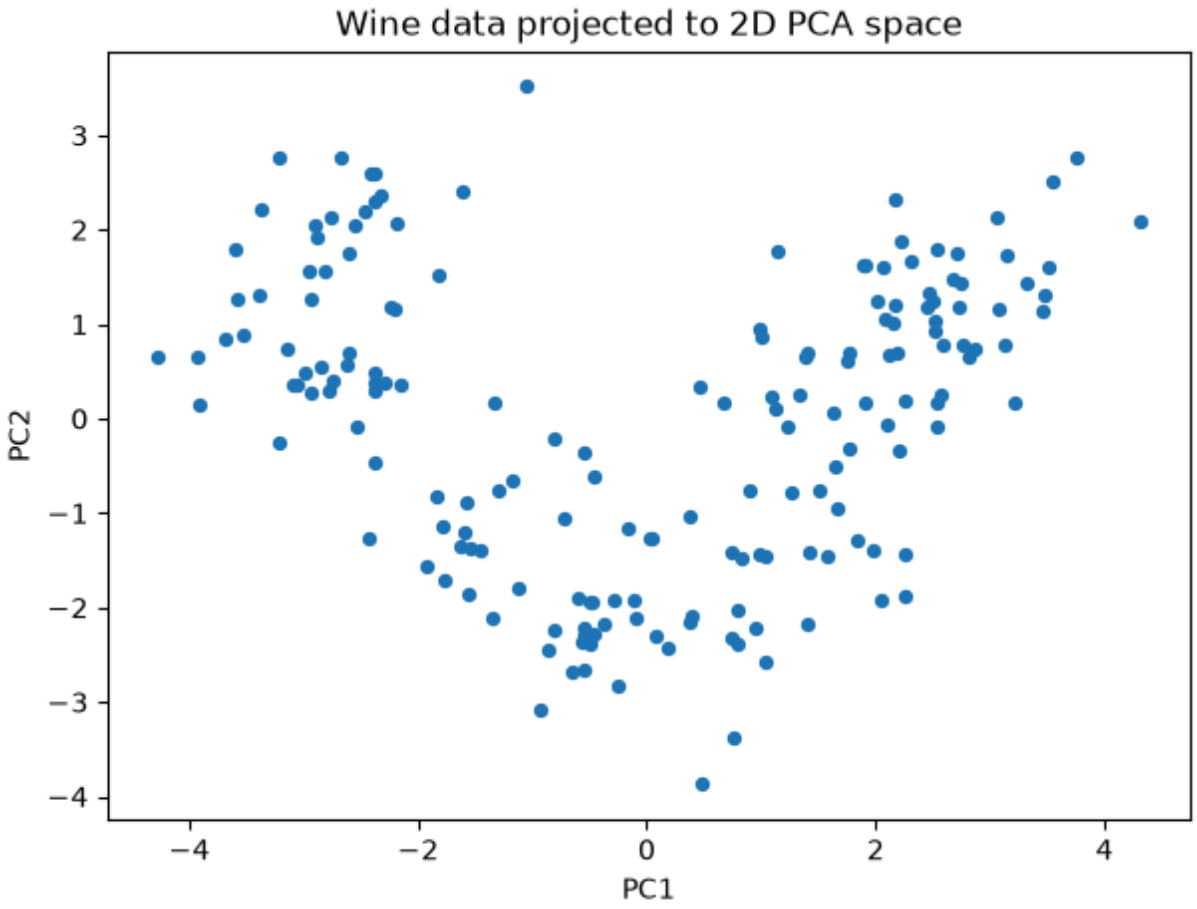
4. PCA for Visualization and Variance Explained

We project data to **2 principal components** for visualization and compute **explained variance ratio (EVR)** to see how much variance is captured.

In [6]:
```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_std)
evr = pca.explained_variance_ratio_
print("Explained variance ratio (2 PCs):", evr, " -> cumulative:", evr.sum()

# Scatter in PCA space (no labels yet, purely unsupervised view)
plt.figure(figsize=(7,5))
plt.scatter(X_pca[:,0], X_pca[:,1], s=18)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Wine data projected to 2D PCA space")
plt.show()
```

```
Explained variance ratio (2 PCs): [0.36198848 0.1920749 ]  -> cumulative: 0.
5540633835693527
```



5. Choosing the Number of Clusters (k)

We use two standard tools:

- **Elbow method**: plot K-Means **inertia** (within-cluster sum of squares) vs k and look for the elbow.

$$\text{Inertia}(k) = \sum_{i=1}^{k} \sum_{x \in \mathcal{C}_i} \|x - \mu_i\|_2^2, \qquad \mu_i \text{ centroid of } \mathcal{C}_i$$

- **Silhouette**: measures how similar a data point is to its own cluster compared to other clusters. It ranges from -1 to 1, where a higher value indicates better-defined clusters. It mainly balances cohesion (how close points are in a cluster) and separation (how far clusters are from each other).

$$s(x) = \frac{b(x) - a(x)}{\max\{a(x), b(x)\}}, \qquad \begin{aligned} a(x) &: \text{ average distance from } x \text{ to all other po} \\ b(x) &: \text{ average distance from } x \text{ to all points in} \end{aligned}$$

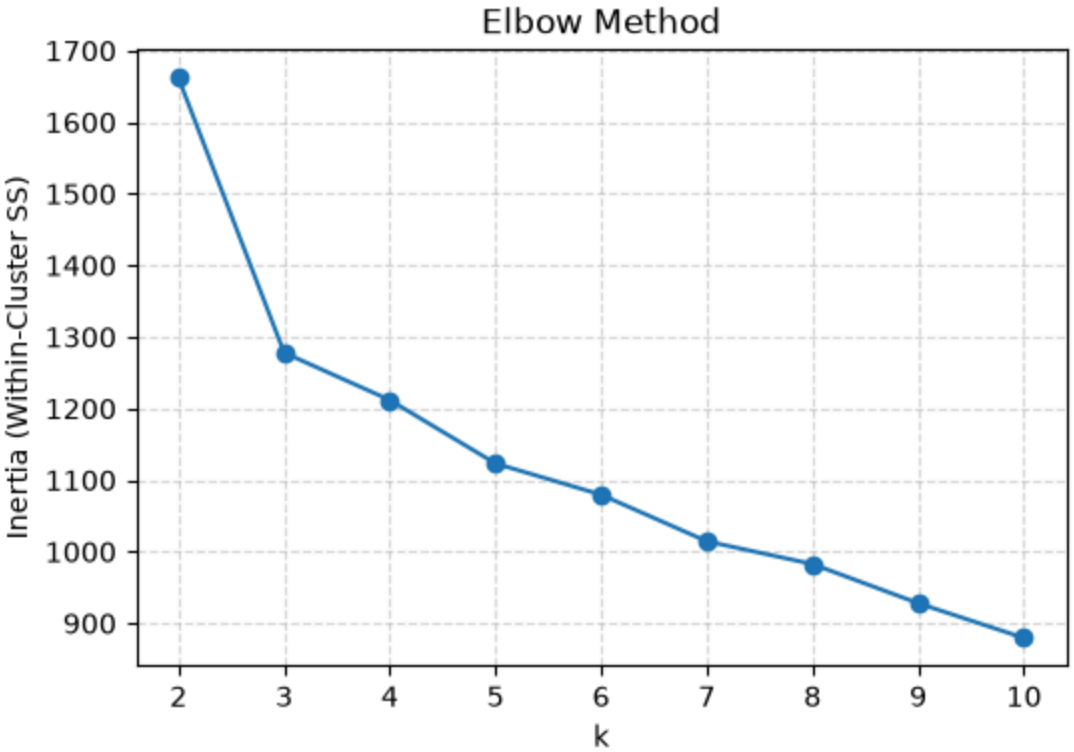
In [7]:
```
ks = range(2, 11)
inertias = []
sil_scores = []
```

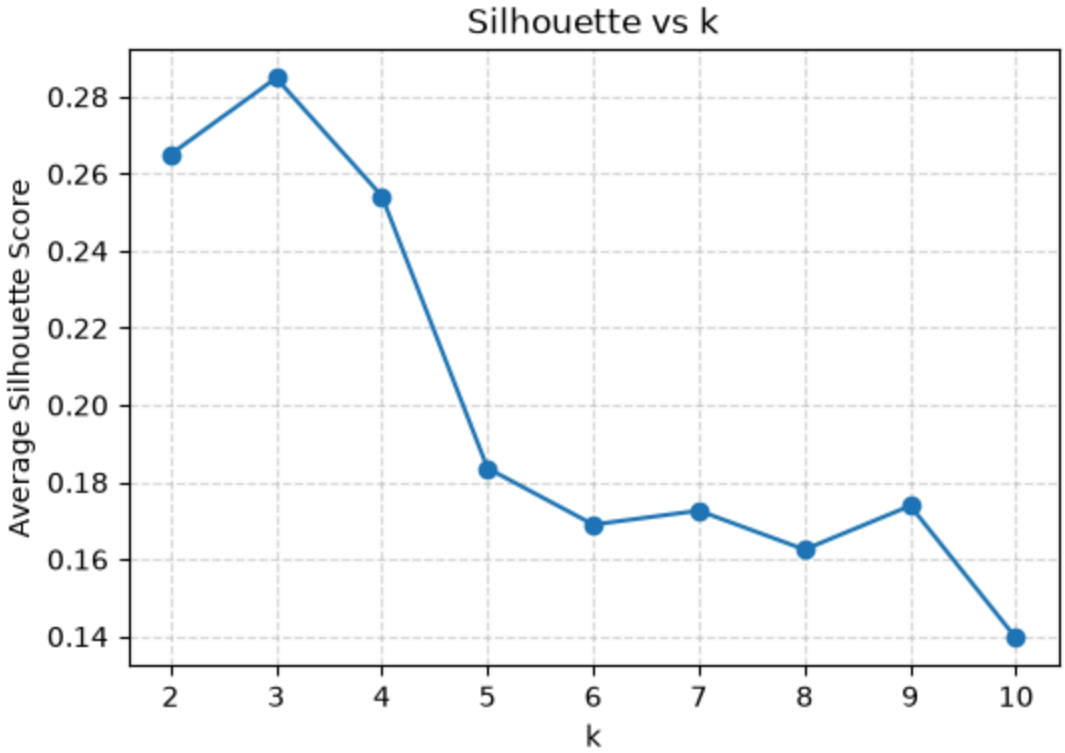
```python
for k in ks:
    km = KMeans(n_clusters=k, n_init="auto", random_state=42)
    km.fit(X_std)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_std, km.labels_))

# Elbow: inertia vs k
plt.figure(figsize=(6,4))
plt.plot(list(ks), inertias, marker="o")
plt.xlabel("k")
plt.ylabel("Inertia (Within-Cluster SS)")
plt.title("Elbow Method")
plt.xticks(list(ks))
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()

# Silhouette vs k
plt.figure(figsize=(6,4))
plt.plot(list(ks), sil_scores, marker="o")
plt.xlabel("k")
plt.ylabel("Average Silhouette Score")
plt.title("Silhouette vs k")
plt.xticks(list(ks))
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()

best_k = ks[int(np.argmax(sil_scores))]
print("Best k by silhouette in this range:", best_k)
```






```
Best k by silhouette in this range: 3
```

In [8]:
```
# Let's check the number of cultivars in the true labels
print("True number of cultivars (classes):", y_true.nunique())
```

```
True number of cultivars (classes): 3
```

6. Fit K-Means

We'll pick the best *k* from the silhouette scan above

In [9]:
```
k = best_k
km = KMeans(n_clusters=k, n_init="auto", random_state=42)
clusters = km.fit_predict(X_std)
```

In [10]:
```
sil_pca = silhouette_score(X_std, clusters)

print("Silhouette score for k={}: {:.4f}".format(k, sil_pca))
```

```
Silhouette score for k=3: 0.2849
```

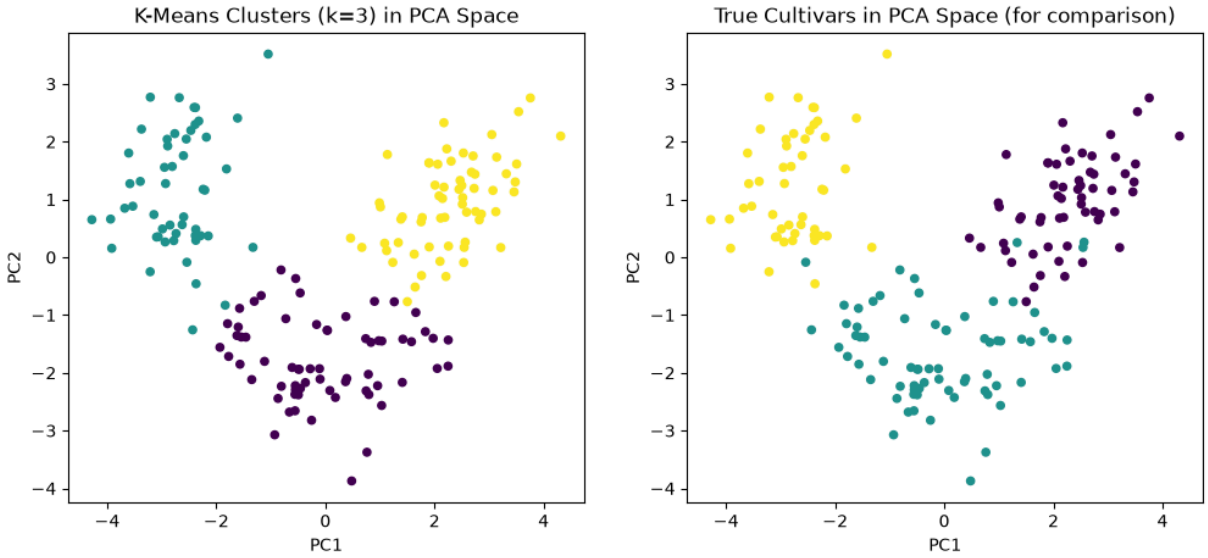
7. Visualize Clusters in PCA Space

We plot K-Means clusters in the 2D PCA projection. We also show the same scatter colored by **true cultivars** (for teaching only) to compare.

In [11]:
```
# Fit PCA again (2D) for clean plotting (already fit above, but okay to keep
pca_plot = PCA(n_components=2, random_state=42).fit(X_std)
X_pca_plot = pca_plot.transform(X_std)
```

```
plt.figure(figsize=(12,5))
# Plot clusters
plt.subplot(1, 2, 1)
plt.scatter(X_pca_plot[:,0], X_pca_plot[:,1], c=clusters, s=20)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(f"K-Means Clusters (k={k}) in PCA Space")

# Plot true labels (teaching only)
plt.subplot(1, 2, 2)
plt.scatter(X_pca_plot[:,0], X_pca_plot[:,1], c=y_true, s=20)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("True Cultivars in PCA Space (for comparison)")
plt.show()
```



8.Does PCA Help Clustering?

Try running K-Means on the first **m** principal components only, then re-evaluate metrics.
Sometimes reducing noise / collinearity helps clustering stability.

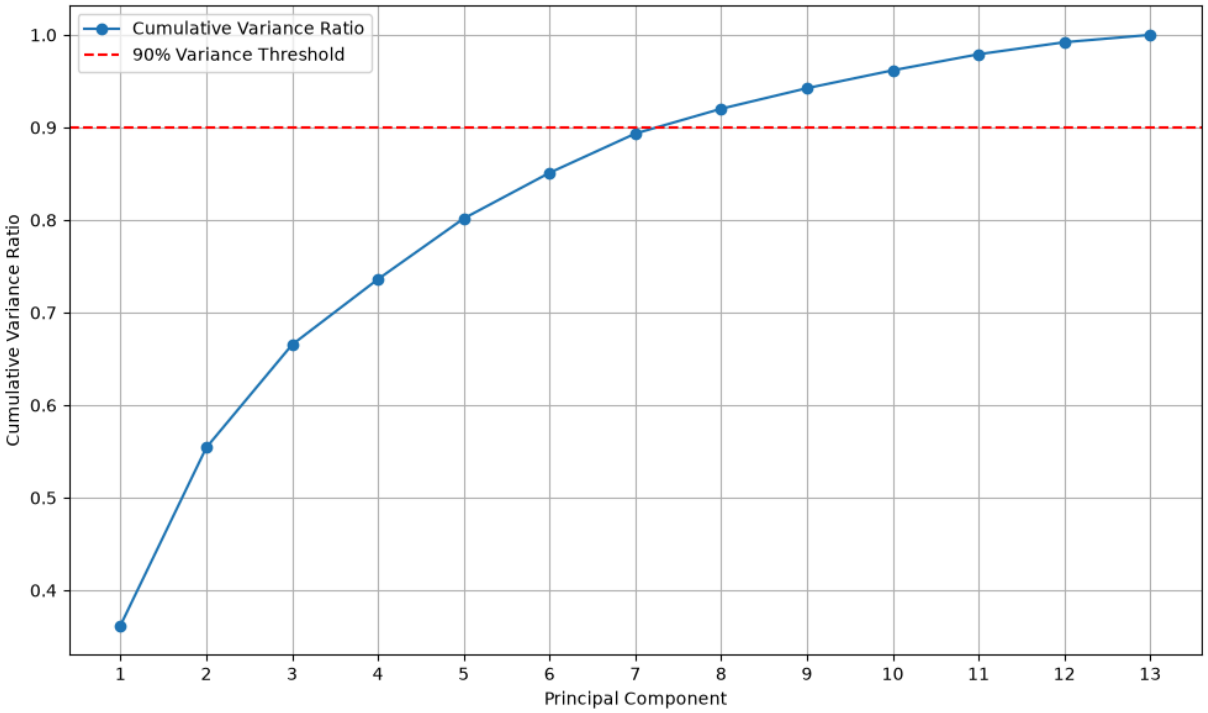
In [ ]:
```
# PCA is useful also to remove noise's contributions

U,S, VT = np.linalg.svd(X_std, full_matrices=False)

# Compute explained variance ratio
EV = (S**2) / (X_std.shape[0] - 1)
EVR = EV / EV.sum()
CVR = np.cumsum(EVR)


# Step 4: Create the scree plot
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(EV) + 1), CVR, marker='o', label='Cumulative Variance
plt.axhline(y=0.90, color='red', linestyle='--', label='90% Variance Thresho
```

```
plt.xlabel('Principal Component')
plt.ylabel('Cumulative Variance Ratio')
plt.xticks(range(1, len(EV) + 1))
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



In [13]:
```
K = np.argmax(CVR >= 0.90) + 1
K # Number of principal components required to retain at least 95% of the va
```

Out[13]:
```
np.int64(8)
```

In [14]:
```
pca_m = PCA(n_components=K, random_state=42)
X_pca_m = pca_m.fit_transform(X_std)
```

In [15]:
```
km_pca = KMeans(n_clusters=k, n_init="auto", random_state=42)
cl_pca = km_pca.fit_predict(X_pca_m)

sil_pca = silhouette_score(X_pca_m, cl_pca)

print(f"K-Means on first {K} PCs -> Silhouette: {sil_pca:.3f}")
```

```
K-Means on first 8 PCs -> Silhouette: 0.315
```

9. Takeaways

- **K-Means** can reveal structure in real chemical data without labels.
- **Scaling** is essential for distance-based clustering.
- **PCA** is useful to **visualize** clusters and sometimes to **improve** clustering by denoising / decorrelating features.

- Use **internal metrics** (Silhouette, CH, DB) to compare settings; use **external labels** only for teaching/validation when available.
- Centroid inspection in **original units** helps translate clusters into **actionable insights** (e.g., which chemical properties define each segment).